

Laborator 2– Algoritmi

1. Implementați o tabelă de dispersie cu rezolvarea coliziunilor prin *adresare deschisă*. Utilizați una dintre metodele de probare liniară, pătratică sau dispersia dublă.

<pre> MAIN //tabelă de dispersie ca vector de întregi Array T[...] READ m //lungime tabelă HASH_INIT(T, m) PRINT "Dati numere pana la citirea lui 0" READ k //cheia de inserat //j e locatia (slotul) unde se afla o cheia k WHILE k<>0 DO j := HASH_INSERT(T, k, m) PRINT "Cheia inserata la locatia"+j READ k //urmatoarea cheia END WHILE //tiparire continut tabelă de dispersie HASH_PRINT(T, m) PRINT "Cautare cheia : " READ k //cheia de cautat j := HASH_SEARCH(T, k, m) IF j > -1 THEN PRINT "Cheia gasita la locatia"+j ELSE PRINT "Cheia nu a fost gasita" END IF PRINT "Stergere cheia: " READ k //cheia de sters j := HASH_DELETE(T, k, m) IF j > -1 THEN PRINT "Cheie stearsa de la locatia"+j HASH_PRINT(T, m) ELSE PRINT "Cheia nu a fost stearsa" END IF END MAIN </pre>	<pre> HASH_INIT(T[], M) //tabelă este T[0..M-1] FOR j := 0 TO M - 1 T[j] := -1 END FOR END HASH_INIT HASH_PRIM(K, M) RETURN K MOD M END HASH_PRIM HASH_PROBEF(K, I, M) //functia de probare liniara RETURN (HASH_PRIM(K, M)+I) MOD M END HASH_PROBEF HASH_INSERT(T[], K, M) i := 0 DO j:= HASH_PROBEF(K, i, M) IF T[j] == -1 THEN T[j] := K RETURN j ELSE i := i + 1 END IF WHILE i <> M RETURN -1 //flag pt eroare -tabelă e plina END HASH_INSERT HASH_SEARCH(T[], K, M) i := 0 DO j := HASH_PROBEF(K, i, M) IF T[j] == K THEN RETURN j END IF i := i + 1 WHILE i <> M AND T[j] <> -1 RETURN -1 //flag pt eroare-cheie negasita END HASH_SEARCH </pre>
--	--

<pre> HASH_DELETE(T[], K, M) j := HASH_SEARCH(T, K, M) IF j > -1 THEN T[j] := -1 RETURN j ELSE RETURN -1 END IF END HASH_DELETE </pre>	<pre> HASH_PRINT(T[], M) FOR j := 0 TO M - 1 PRINT T[j] END FOR END HASH_PRINT </pre>
---	---

2. Implementați o tabelă de dispersie cu rezolvarea coliziunilor prin înlănțuire, utilizând o listă dublu înlănțuită. Veți utiliza implementările listei dublu înlănțuite realizate la laboratorul precedent prin simpla includere a fisierului cu extensia .h care definește operațiile specifice pe o asemenea structură. Pentru această tabelă de dispersie implementați operațiile de INSERT, SEARCH și DELETE. Dacă m este dimensiunea sirului care salvează tabela, atunci puteți considera ca și funcție de dispersie $h(k) = k \bmod m$.

Modul de implementare a unei tabele de dispersie:

<pre> typedef struct nod_lista { int cheie; struct nod_lista *next, *prev; } t_nod_lista; typedef struct { t_nod_lista *head; } t_lista; </pre>	Obs: Dacă definiția unei liste dublu înlănțuită este cea de alături atunci o tabelă de dispersie care utilizează liste dublu înlănțuite este un vector de astfel de liste care de declară astfel: <pre> t_lista T[100]; //tabela de dispersie </pre>
--	---

Pseudocod:

<pre> MAIN LISTA T[...] PRINT "Funcția de dispersie h(k)=k mod m" READ m //lungimea tabelii de dispersie INIT_HASHTABLE(T, m) PRINT "Dati numere pana la citirea lui 0" READ k //cheie de inserat in tabela ↓ (continuare funcție MAIN) </pre>	<pre> INIT_HASHTABLE(LISTA T[], M) FOR j := 0 TO M-1 MAKENULL(T[j]) //de la liste END FOR END INIT_HASHTABLE HASH_FUNCTION(K, M) RETURN K MOD M END HASH_FUNCTION </pre>
--	--

<pre> WHILE k<>0 DO //inserare cheie k la slotul j j := CHAINED_HASH_INSERT(T, k, m) PRINT "Tabela actualizata: " PRINT_HASHTABLE(T, m) READ k //citeste urmatoarea cheie END WHILE PRINT "Cautare cheie: " READ k //Cheia de cautat in tabela //returneaza locatia j unde s-a gasit cheia j := CHAINED_HASH_SEARCH(T, k, m) IF j > -1 THEN PRINT "Cheia gasita la locatia"+j ELSE PRINT "Cheia nu a fost gasita" END IF PRINT "Cheia pentru stergere: " READ k j := CHAINED_HASH_DELETE(T, k, m) IF j > -1 THEN PRINT "Cheie stearsa de la locatia"+j PRINT_HASHTABLE(T, m) ELSE PRINT "Cheia nu a fost gasita" END IF END MAIN </pre>	<pre> CHAINED_HASH_INSERT(LISTA T[], K, M) Nod_lista x //element de inserat j := HASH_FUNCTION(K, M) Allocate mem. for x x.key := K LIST_INSERT(T[j], x) RETURN j END CHAINED_HASH_INSERT PRINT_HASHTABLE(LISTA T[], M) FOR j := 0 TO M-1 IF T[j].head.next <> NULL THEN PRINT "T[j]: " LIST_PRINT(T[j]) END IF END FOR END PRINT_HASHTABLE </pre>
<pre> CHAINED_HASH_SEARCH(LISTA T[], K, M) Nod_lista x //element de cautat j := HASH_FUNCTION(K, M) x := LIST_SEARCH(T[j], K) IF x <> NULL THEN RETURN j ELSE RETURN -1 END IF END CHAINED_HASH_SEARCH </pre>	<pre> CHAINED_HASH_DELETE(LISTA T[], K, M) Nod_lista x //element de sters j := HASH_FUNCTION(K, M) x := LIST_SEARCH(T[j], K) IF x <> NULL THEN LIST_DELETE(T[j], x) RETURN j ELSE RETURN -1 END IF END CHAINED_HASH_DELETE </pre>

Temă. Probleme optionale

1. Intr-o tabelă de dispersie cu rezolvarea coliziunilor prin înlănțuire, presupunem că fiecare slot conține un flag și una din următoarele: (i) un element și un pointer, sau (ii) doi pointeri. Ideea este să legăm între ele (intr-o listă simplă sau dublu înlănțuită) sloturile libere din tabela de dispersie.

Implementati operațiile de alocare si dealocare de de memorie pentru elemente intr-o tabela de dispersie ca si cea descrisă mai sus.

2. Dorim să implementăm un dictionar folosind o adresare directă pe un sir de dimensiune foarte mare. La inceput, sirul contine garbage, iar initializarea sirului nu este fezabilă datorită dimensiunii foarte mari a acestuia. Implementati un asemenea dictionar, astfel incat fiecare obiect salvat să ocupe un spatiu de memorie $O(1)$, operațiile de SEARCH, INSERT, DELETE să se realizeze in $O(1)$, iar initializarea, la fel in $O(1)$. Se va utiliza un sir auxiliar care să functioneze similar unei stive, a cărei mărime să fie egală cu numărul de chei stocate la un moment dat in dictionar pentru a ne ajuta să determinăm dacă o intrare in sirul de dimensiune foarte mare este valida sau nu.